

Pontifícia Universidade Católica de Campinas

Estruturas e Recuperação de Dados II



GRAFOS

Dijkstra



Prof. Me. Darcy H.F.R. Moreira



Observações:

DFS e BFS

Vértice descoberto – Momento em que ele é **adicionado** na estrutura.

- Controle do algoritmo;
- **Marcação** para que ele não entre novamente na estrutura.

Ordem de visitação: ordem em que os vértices são **removidos** da estrutura.

- Ordem de **processamento**

Observações:

DFS e BFS

Vértice descoberto – Momento em que ele é **adicionado** na estrutura.

- Controle do algoritmo;
- **Marcação** para que ele não entre novamente na estrutura.

Ordem de visitação: ordem em que os vértices são **removidos** da estrutura.

- Ordem de **processamento**

BFS:

enfileirou → descobriu (marco)

desenfileirou → visitou/processou (remove)

DFS:

empilhou → descobriu (marcou)

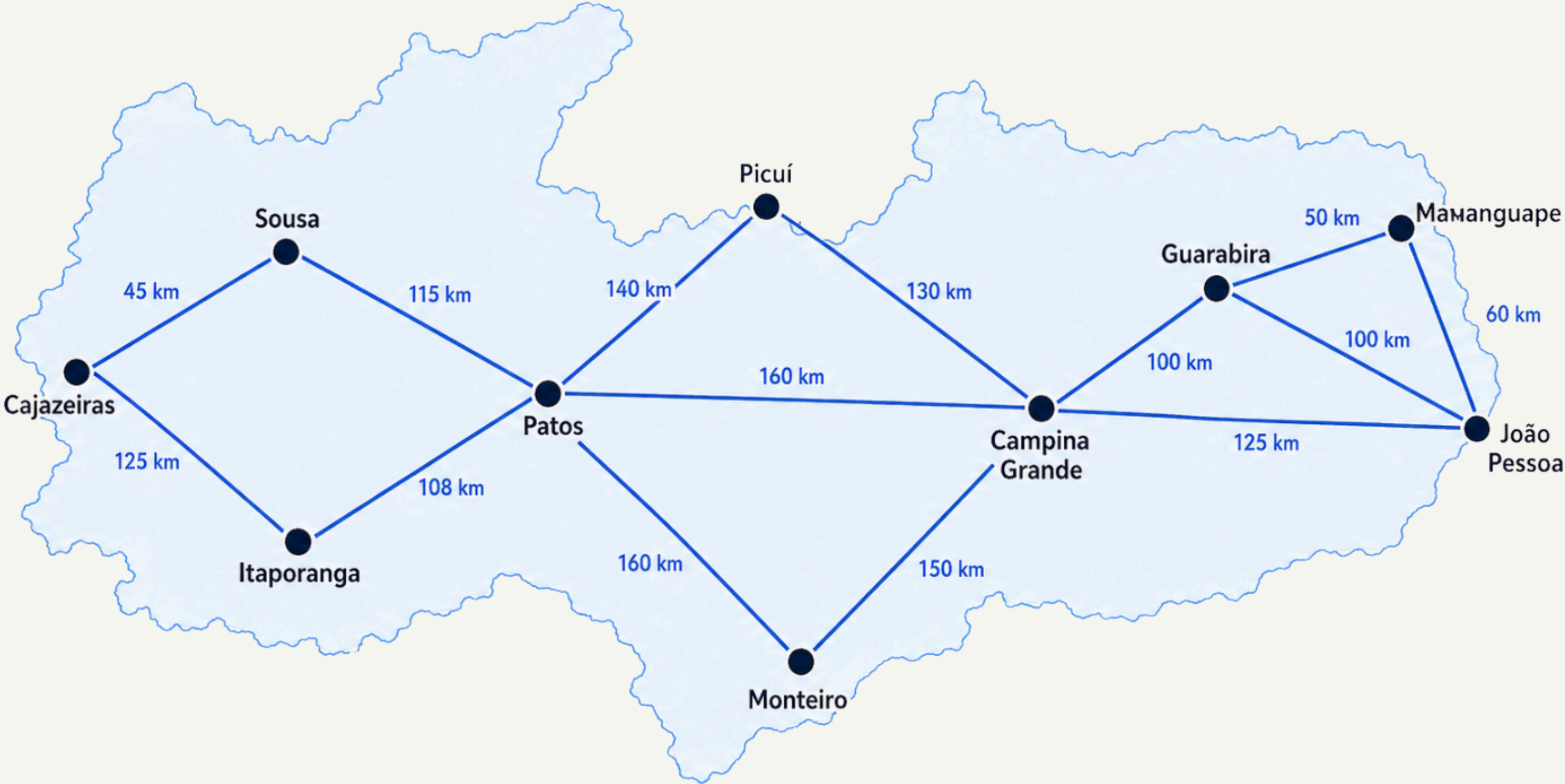
desempilhou → visitou/processou (removeu)

GRAFOS NÃO PONDERADOS

sem pesos nas arestas

Busca em LARGURA (BFS): fornece o **menor caminho** (em número de arestas)

Pergunta: Qual a maneira mais rápida para ir de Cajazeiras à João Pessoa?



Como encontramos caminhos de custo (peso) mínimo?

Floyd-Warshall: encontra os menores caminhos entre todos os pares de vértices do grafo.

Bellman-Ford: encontra os menores caminhos a partir de um vértice de origem para todos os demais, permitindo inclusive arestas com pesos negativos, desde que não existam ciclos negativos.

Dijkstra: encontra os menores caminhos a partir de um vértice de origem para todos os demais, considerando grafos com pesos não negativos.

Como encontramos caminhos de custo (peso) mínimo?

Floyd-Warshall: encontra os menores caminhos entre todos os pares de vértices do grafo.

Bellman-Ford: encontra os menores caminhos a partir de um vértice de origem para todos os demais, permitindo inclusive arestas com pesos negativos, desde que não existam ciclos negativos.

Dijkstra: encontra os menores caminhos a partir de um vértice de origem para todos os demais, considerando grafos com pesos não negativos.

Algoritmo de Dijkstra

Criado pelo cientista da computação holandês Edsger W. Dijkstra.

Encontrar os caminhos de **menor custo** a partir de um vértice de **origem** até os demais vértices de um grafo.

Algoritmo de Dijkstra

Criado pelo cientista da computação holandês Edsger W. Dijkstra.

Encontrar os caminhos de **menor custo** a partir de um vértice de **origem** até os demais vértices de um grafo.

- grafos ponderados;
- grafos com pesos **não negativos**.

Algoritmo de Dijkstra

Criado pelo cientista da computação holandês Edsger W. Dijkstra.

Encontrar os caminhos de **menor custo** a partir de um vértice de **origem** até os demais vértices de um grafo.

- grafos ponderados;
- grafos com pesos **não negativos**.

A cada etapa, verifica se é possível tentar melhorar a distância conhecida até um vértice. Esse processo de atualização das distâncias chamado de **relaxamento**.

Algoritmo de Dijkstra

1. Marcamos todos os vértices como **não-explorados**

Algoritmo de Dijkstra

1. Marcamos todos os vértices como **não-explorados**
2. Atribuimos a todos os vértices uma **tentativa** de distância de início

Vértice de origem → **$d = 0$**
Demais vértices → **$d = \infty$**

Algoritmo de Dijkstra

1. Marcamos todos os vértices como **não-explorados**
2. Atribuimos a todos os vértices uma **tentativa** de distância de início

Vértice de origem → **$d = 0$**
Demais vértices → **$d = \infty$**
3. Escolhemos como vértice atual o vértice **não explorado** com a **menor distância** conhecida.
 - a. Marcamos como **explorado** e sua distância é definitiva.

Algoritmo de Dijkstra

1. Marcamos todos os vértices como **não-explorados**
2. Atribuimos a todos os vértices uma **tentativa** de distância de início

Vértice de origem → **$d = 0$**
Demais vértices → **$d = \infty$**
3. Escolhemos como vértice atual o vértice **não explorado** com a **menor distância** conhecida.
 - a. Marcamos como **explorado** e sua distância é definitiva.
4. Para o vértice atual, analisamos seus vizinhos ainda **não-explorados**:
 - a. Para **cada vizinho**, calculamos uma possível **nova** distância:
 $\text{nova distância} = \text{distância do atual} + \text{peso da aresta}$
 - b. Se a nova distância for **MENOR** que a antiga: atualizamos a **distância** e o **pai**, que é o vértice atual

Algoritmo de Dijkstra

1. Marcamos todos os vértices como **não-explorados**
2. Atribuimos a todos os vértices uma **tentativa** de distância de início

Vértice de origem → **$d = 0$**
Demais vértices → **$d = \infty$**

3. Escolhemos como vértice atual o vértice **não explorado** com a **menor distância** conhecida.
 - a. Marcamos como **explorado** e sua distância é definitiva.
4. Para o vértice atual, analisamos seus vizinhos ainda **não-explorados**:
 - a. Para **cada vizinho**, calculamos uma possível **nova** distância:
 $\text{nova distância} = \text{distância do atual} + \text{peso da aresta}$
 - b. Se a nova distância for **MENOR** que a antiga: atualizamos a **distância** e o **pai**, que é o vértice atual

Repetimos o processo até não haver mais vértices não explorados alcançáveis.

